

ROS 2 Jazzy + Gazebo Harmonic

직접 제작한 STL 기반 Ackermann 차량 SLAM 구축 워크플로우

STL -> URDF/Xacro -> ros2_control -> Gazebo -> LiDAR -> SLAM Toolbox

문서 목표

직접 설계한 STL 메시를 ROS 2 Jazzy와 Gazebo Harmonic에 올리고, 전륜 조향/후륜 구동 Ackermann 차량을 구성한 뒤 2D LiDAR와 오도메트리를 이용해 SLAM Toolbox로 지도를 작성하는 전체 절차를 단계별로 설명한다.

작성 기준: Ubuntu 24.04 / ROS 2 Jazzy / Gazebo Harmonic

문서 생성일: 2026-07-30

목차

1. 전체 시스템 구성
2. STL 과 차량 치수 준비
3. ROS 패키지 설치
4. 워크스페이스와 패키지 생성
5. URDF/Xacro 링크와 조인트
6. 충돌체, 질량, 관성
7. gz_ros2_control 구성
8. Ackermann 컨트롤러 설정
9. 2D LiDAR 구성
10. Gazebo 월드와 브리지
11. Launch 구성
12. 단계별 검증
13. 오도메트리와 TF
14. SLAM Toolbox
15. RViz 및 지도 작성
16. 지도 저장
17. 오류 진단
18. 최종 체크리스트
19. 참고자료

1. 전체 시스템 구성

이 문서는 전륜 조향, 후륜 구동 구조를 기준으로 한다. 최종적으로 키보드 또는 Nav2 가 보낸 속도 명령을 Ackermann 컨트롤러가 각 조인트 명령으로 변환하고, Gazebo 가 물리 운동과 LiDAR 데이터를 생성하며, SLAM Toolbox 가 /scan 과 오도메트리를 이용해 /map 을 생성한다.

전체 데이터 흐름

```

teleop / Nav2
      | geometry_msgs/TwistStamped
      v
/cmd_vel
      |
      v
ackermann_steering_controller
  |-- rear wheel velocity commands
  |-- front steering position commands
  |-- odometry
  |-- odom -> base_footprint TF
      |
      v
      gz_ros2_control
      |
      v
      Gazebo vehicle model

Gazebo GPU LiDAR -> ros_gz_bridge -> /scan -> slam_toolbox
slam_toolbox -> /map and map -> odom TF
  
```

최종 TF 트리

```

map
|-- odom
    |-- base_footprint
        |-- base_link
            |-- lidar_link
            |-- front_left_knuckle_link
            |-- front_left_wheel_link
            |-- front_right_knuckle_link
            |-- front_right_wheel_link
            |-- rear_left_wheel_link
            |-- rear_right_wheel_link
  
```

TF 발행 책임

map -> odom 은 SLAM Toolbox 가, odom -> base_footprint 는 Ackermann 컨트롤러가, base_footprint 이하의 고정/관절 TF 는 robot_state_publisher 가 담당하게 한다. 동일한 TF 를 둘 이상의 노드가 중복 발행하면 지도가 흔들리거나 TF 오류가 발생한다.

2. STL 과 차량 치수 준비

권장 STL 분리 수준

움직임이 서로 다른 부품만 별도 STL 로 분리한다. 볼트, 모터 외형, 배터리 케이스처럼 차체에 고정된 부품은 chassis.stl 에 합쳐도 된다.

```

chassis.stl
rear_left_wheel.stl
rear_right_wheel.stl
front_left_knuckle.stl
front_right_knuckle.stl
front_left_wheel.stl
front_right_wheel.stl
  
```

파일	역할	URDF 조인트
chassis.stl	고정 차체 전체	기준 링크
rear_left_wheel.stl	왼쪽 후륜 구동	continuous
rear_right_wheel.stl	오른쪽 후륜 구동	continuous
front_left_knuckle.stl	왼쪽 조향부	revolute

파일	역할	URDF 조인트
front_right_knuckle.stl	오른쪽 조향부	revolute
front_left_wheel.stl	왼쪽 앞바퀴 굴림	continuous, passive
front_right_wheel.stl	오른쪽 앞바퀴 굴림	continuous, passive

CAD 에서 반드시 측정할 값

파라미터	의미	측정 기준
wheelbase	축거	앞 조향축 중심과 뒤 구동축 중심 사이 거리
traction_track_width	후륜 윤거	좌우 후륜 중심 사이 거리
steering_track_width	전륜 윤거	좌우 조향축 중심 사이 거리
traction_wheels_radius	구동 바퀴 반지름	충돌 원통의 실제 반지름과 동일
wheel_width	바퀴 폭	원통 충돌체 length
steering_limit	최대 조향각	킹핀 기준 회전 가능한 최대 각도

예시 치수 - 반드시 실제 CAD 값으로 교체

```
wheelbase: 0.42
traction_track_width: 0.34
steering_track_width: 0.34
traction_wheels_radius: 0.065
wheel_width: 0.035
steering_limit: 0.55
```

좌표계와 원점

ROS 차량 좌표계
X: 차량 전방
Y: 차량 왼쪽
Z: 차량 위쪽

- 후륜 STL 원점: 바퀴 회전축 중앙
- 앞바퀴 STL 원점: 바퀴 회전축 중앙
- 너클 STL 원점: 킹핀 또는 조향축 중심
- 차체 STL 원점: 차량 중심 또는 뒤축 중심 중 하나로 일관되게 설정

STL 단위 주의

STL 에는 단위 정보가 없다. CAD 에서 mm 로 내보낸 메시를 URDF 에서 사용할 때는 일반적으로 mesh scale 을 0.001 0.001 0.001 로 지정한다. 모든 STL 에 같은 단위 규칙을 적용해야 한다.

3. ROS 패키지 설치

Ubuntu 24.04 와 ROS 2 Jazzy 가 설치되어 있다는 전제에서 필요한 패키지를 설치한다. Jazzy 에서는 Gazebo Classic 용 gazebo_ros_pkgs 가 아니라 ros_gz 와 gz_ros2_control 계열을 사용한다.

```
sudo apt update

sudo apt install \
  ros-jazzy-ros-gz \
  ros-jazzy-gz-ros2-control \
  ros-jazzy-ros2-controllers \
  ros-jazzy-xacro \
  ros-jazzy-robot-state-publisher \
  ros-jazzy-joint-state-publisher-gui \
  ros-jazzy-slam-toolbox \
  ros-jazzy-rviz2 \
  ros-jazzy-teleop-twist-keyboard \
  ros-jazzy-tf2-tools \
  ros-jazzy-nav2-map-server
```

설치 확인

```
source /opt/ros/jazzy/setup.bash

ros2 pkg prefix ros_gz_sim
ros2 pkg prefix gz_ros2_control
ros2 pkg prefix ackermann_steering_controller
ros2 pkg prefix slam_toolbox
```

4. 워크스페이스와 패키지 생성

```
mkdir -p ~/ackermann_ws/src
cd ~/ackermann_ws/src

ros2 pkg create \
  --build-type ament_cmake \
  ackermann_car_description
```

추천 패키지 구조

```
ackermann_ws/src/ackermann_car_description/
|-- CMakeLists.txt
|-- package.xml
|-- meshes/
|   |-- chassis.stl
|   |-- rear_left_wheel.stl
|   |-- rear_right_wheel.stl
|   |-- front_left_knuckle.stl
|   |-- front_right_knuckle.stl
|   |-- front_left_wheel.stl
|   |-- front_right_wheel.stl
|-- urdf/
|   |-- ackermann_car.urdf.xacro
|   |-- materials.xacro
|   |-- inertial_macros.xacro
|   |-- ros2_control.xacro
|   |-- lidar.xacro
|-- config/
|   |-- controllers.yaml
|   |-- bridge.yaml
|   |-- slam.yaml
|   |-- ackermann_car.rviz
|-- launch/
|   |-- display.launch.py
|   |-- simulation.launch.py
|   |-- slam.launch.py
|-- worlds/
|   |-- slam_world.sdf
```

CMakeLists.txt 에 추가

```
install(
  DIRECTORY
  launch
  urdf
  meshes
  config
  worlds
  DESTINATION share/${PROJECT_NAME}
)

cd ~/ackermann_ws
colcon build --symlink-install
source install/setup.bash
```

5. URDF/Xacro 링크와 조인트

기본 링크 트리

```
base_footprint
|-- base_link
|   |-- rear_left_wheel_link
|   |-- rear_right_wheel_link
|   |-- front_left_knuckle_link
|   |-- front_left_wheel_link
|   |-- front_right_knuckle_link
|   |-- front_right_wheel_link
|-- lidar_link
```

base_footprint 와 base_link

```
<link name="base_footprint"/>

<joint name="base_footprint_joint" type="fixed">
  <parent link="base_footprint"/>
  <child link="base_link"/>
  <origin xyz="0 0 0.12" rpy="0 0 0"/>
</joint>
```

base_footprint 는 바닥면에 투영된 좌표계이고 base_link 는 실제 차체 기준 좌표계다. 위 예시의 0.12 는 차량 바닥에서 base_link 원점까지의 높이이므로 실제 설계값으로 바뀌어야 한다.

차체 링크

```
<link name="base_link">
  <visual>
    <geometry>
      <mesh
        filename="package://ackermann_car_description/meshes/chassis.stl"
        scale="0.001 0.001 0.001"/>
      </geometry>
    </visual>

    <collision>
      <geometry>
        <box size="0.55 0.34 0.16"/>
      </geometry>
    </collision>

    <inertial>
      <mass value="5.0"/>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <inertia ixx="0.06" ixy="0" ixz="0"
        iyy="0.12" iyz="0" izz="0.15"/>
    </inertial>
  </link>
```

후륜 구동 조인트

```
<joint name="rear_right_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="rear_right_wheel_link"/>
  <origin xyz="-0.20 -0.17 -0.06" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <dynamics damping="0.1" friction="0.0"/>
</joint>

<joint name="rear_left_wheel_joint" type="continuous">
  <parent link="base_link"/>
  <child link="rear_left_wheel_link"/>
  <origin xyz="-0.20 0.17 -0.06" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <dynamics damping="0.1" friction="0.0"/>
</joint>
```

전륜 조향 조인트

```
<joint name="front_right_steering_joint" type="revolute">
  <parent link="base_link"/>
  <child link="front_right_knuckle_link"/>
  <origin xyz="0.22 -0.17 -0.06" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-0.55" upper="0.55"
    effort="30.0" velocity="2.0"/>
  <dynamics damping="1.0" friction="0.1"/>
</joint>

<joint name="front_left_steering_joint" type="revolute">
  <parent link="base_link"/>
  <child link="front_left_knuckle_link"/>
  <origin xyz="0.22 0.17 -0.06" rpy="0 0 0"/>
  <axis xyz="0 0 1"/>
  <limit lower="-0.55" upper="0.55"
    effort="30.0" velocity="2.0"/>
  <dynamics damping="1.0" friction="0.1"/>
</joint>
```

앞바퀴 굴림 조인트

```
<joint name="front_right_wheel_joint" type="continuous">
  <parent link="front_right_knuckle_link"/>
  <child link="front_right_wheel_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <dynamics damping="0.05" friction="0.0"/>
</joint>

<joint name="front_left_wheel_joint" type="continuous">
  <parent link="front_left_knuckle_link"/>
  <child link="front_left_wheel_link"/>
  <origin xyz="0 0 0" rpy="0 0 0"/>
  <axis xyz="0 1 0"/>
  <dynamics damping="0.05" friction="0.0"/>
</joint>
```

앞바퀴와 너클을 나누는 이유

앞바퀴는 조향축을 중심으로 좌우로 꺾이는 운동과 바퀴 축을 중심으로 굴러가는 운동을 동시에 해야 한다. 하나의 STL/링 크로 합치면 두 개의 독립 회전 자유도를 표현할 수 없다.

6. 충돌체, 질량, 관성

시각 모델에는 원본 STL 을 사용해도 되지만, 충돌 판정에는 박스와 원통 같은 단순 형상을 사용하는 편이 계산량과 물리 안정성 면에서 유리하다.

바퀴 충돌체 예시

```
<link name="rear_right_wheel_link">
  <visual>
    <geometry>
      <mesh
        filename="package://ackermann_car_description/meshes/rear_right_wheel.stl"
        scale="0.001 0.001 0.001"/>
      </geometry>
    </visual>

    <collision>
      <origin xyz="0 0 0" rpy="1.5708 0 0"/>
      <geometry>
        <cylinder radius="0.065" length="0.035"/>
      </geometry>
    </collision>

    <inertial>
      <mass value="0.25"/>
      <inertia ixx="0.0004" ixy="0" ixz="0"
        iyy="0.0007" iyz="0" izz="0.0004"/>
    </inertial>
  </link>
```

물리 안정성 점검

- 차체와 바퀴 충돌체가 서로 깊게 겹치지 않는지 확인
- 차체 충돌체가 바닥에 닿거나 바퀴 아래로 내려오지 않는지 확인
- 바퀴 원통 축이 실제 회전축과 같은 방향인지 확인
- 질량이 0 이거나 관성값이 비정상적으로 작지 않은지 확인
- 무게중심이 너무 높아 차량이 쉽게 전복되지 않는지 확인

7. gz_ros2_control 구성

후륜에는 velocity command interface 를, 전륜 조향에는 position command interface 를 제공한다. 앞바퀴 굴림 조인트는 명령 없이 state interface 만 제공하거나 ros2_control 밖의 수동 조인트로 두어도 된다.

```
<ros2_control name="GazeboSimSystem" type="system">
  <hardware>
    <plugin>gz_ros2_control/GazeboSimSystem</plugin>
  </hardware>

  <joint name="rear_right_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-30.0</param>
      <param name="max">30.0</param>
    </command_interface>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>

  <joint name="rear_left_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-30.0</param>
      <param name="max">30.0</param>
    </command_interface>
    <state_interface name="position"/>
    <state_interface name="velocity"/>
  </joint>

  <joint name="front_right_steering_joint">
    <command_interface name="position">
      <param name="min">-0.55</param>
      <param name="max">0.55</param>
    </command_interface>
  </joint>
```

```

</command_interface>
<state_interface name="position"/>
<state_interface name="velocity"/>
</joint>

<joint name="front_left_steering_joint">
  <command_interface name="position">
    <param name="min">-0.55</param>
    <param name="max">0.55</param>
  </command_interface>
  <state_interface name="position"/>
  <state_interface name="velocity"/>
</joint>

<joint name="front_right_wheel_joint">
  <state_interface name="position"/>
  <state_interface name="velocity"/>
</joint>

<joint name="front_left_wheel_joint">
  <state_interface name="position"/>
  <state_interface name="velocity"/>
</joint>
</ros2_control>

```

Gazebo 플러그인

```

<gazebo>
  <plugin
    filename="libgz_ros2_control-system.so"
    name="gz_ros2_control::GazeboSimROS2ControlPlugin">

    <parameters>
      $(find ackermann_car_description)/config/controllers.yaml
    </parameters>

    <position_proportional_gain>0.5</position_proportional_gain>

    <ros>
      <remapping>
        /ackermann_steering_controller/tf_odometry:=/tf
      </remapping>
      <remapping>
        /ackermann_steering_controller/reference:=/cmd_vel
      </remapping>
    </ros>
  </plugin>
</gazebo>

```

토픽 이름 확인

컨트롤러의 입력 토픽은 컨트롤러 버전과 namespace 설정에 따라 `/ackermann_steering_controller/reference` 형 태로 나타날 수 있다. remapping 을 쓰기 전에 `ros2 topic list` 와 `ros2 topic info` 로 실제 토픽을 확인한다.

8. Ackermann 컨트롤러 설정

config/controllers.yaml

```

controller_manager:
  ros__parameters:
    update_rate: 100

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    ackermann_steering_controller:
      type: ackermann_steering_controller/AckermannSteeringController

ackermann_steering_controller:
  ros__parameters:
    reference_timeout: 0.5

    # 순서: 오른쪽, 왼쪽
    traction_joints_names:
      - rear_right_wheel_joint
      - rear_left_wheel_joint

    steering_joints_names:
      - front_right_steering_joint
      - front_left_steering_joint

    wheelbase: 0.42
    traction_track_width: 0.34
    steering_track_width: 0.34
    traction_wheels_radius: 0.065

    base_frame_id: base_footprint

```

```
odom_frame_id: odom
enable_odom_tf: true

open_loop: false
position_feedback: false
reduce_wheel_speed_until_steering_reached: true
velocity_rolling_window_size: 10

pose_covariance_diagonal:
- 0.001
- 0.001
- 0.001
- 0.001
- 0.001
- 0.01

twist_covariance_diagonal:
- 0.001
- 0.001
- 0.001
- 0.001
- 0.001
- 0.01
```

설정	역할	초기 권장
open_loop	명령값 기반 오도메트리 여부	Gazebo 조인트 피드백을 쓰려면 false
position_feedback	바퀴 위치/속도 중 피드백 선택	velocity 상태 사용 시 false
enable_odom_tf	odom -> base TF 발행	true
reference_timeout	명령 단절 시 정지 시간	0.3-0.5 s
update_rate	컨트롤러 업데이트 주기	50-100 Hz

9. 2D LiDAR 구성

LiDAR 링크

```
<link name="lidar_link">
  <visual>
    <geometry>
      <cylinder radius="0.04" length="0.04"/>
    </geometry>
  </visual>
</link>

<joint name="lidar_joint" type="fixed">
  <parent link="base_link"/>
  <child link="lidar_link"/>
  <origin xyz="0.12 0 0.22" rpy="0 0 0"/>
</joint>
```

Gazebo GPU LiDAR

```
<gazebo reference="lidar_link">
  <sensor name="front_lidar" type="gpu_lidar">
    <always_on>true</always_on>
    <visualize>true</visualize>
    <update_rate>10</update_rate>

    <topic>scan</topic>
    <gz_frame_id>lidar_link</gz_frame_id>

    <lidar>
      <scan>
        <horizontal>
          <samples>720</samples>
          <resolution>1</resolution>
          <min_angle>-3.14159</min_angle>
          <max_angle>3.14159</max_angle>
        </horizontal>
      </scan>
      <range>
        <min>0.10</min>
        <max>12.0</max>
        <resolution>0.01</resolution>
      </range>
    </lidar>
  </sensor>
</gazebo>
```

LiDAR 높이

레이저 평면이 차체 자체나 바퀴를 가로지르지 않도록 lidar_link 높이를 잡는다. SLAM 테스트 월드의 벽 높이는 LiDAR 스캔 높이보다 높아야 한다.

10. Gazebo 월드와 브리지

월드에서 필요한 시스템 플러그인

```
<plugin filename="gz-sim-physics-system"
  name="gz::sim::systems::Physics"/>

<plugin filename="gz-sim-user-commands-system"
  name="gz::sim::systems::UserCommands"/>

<plugin filename="gz-sim-scene-broadcaster-system"
  name="gz::sim::systems::SceneBroadcaster"/>

<plugin filename="gz-sim-sensors-system"
  name="gz::sim::systems::Sensors">
  <render_engine>ogre2</render_engine>
</plugin>
```

SLAM 월드에는 빈 정사각형보다 길이가 다른 벽, 기둥, 모서리, 통로처럼 방향을 구별할 수 있는 특징을 포함해야 한다. 완전히 대칭적인 환경은 scan matching 과 loop closure 를 어렵게 만든다.

ros_gz_bridge 설정

config/bridge.yaml

```
- ros_topic_name: /clock
  gz_topic_name: /clock
  ros_type_name: roscpp_msgs/msg/Clock
  gz_type_name: gz.msgs.Clock
  direction: GZ_TO_ROS

- ros_topic_name: /scan
  gz_topic_name: /scan
  ros_type_name: sensor_msgs/msg/LaserScan
  gz_type_name: gz.msgs.LaserScan
  direction: GZ_TO_ROS
```

joint_states, controller 입력, odometry 와 TF 는 gz_ros2_control 이 ROS 쪽에서 직접 제공하므로 일반적으로 별도의 ros_gz_bridge 항목이 필요하지 않다. 반면 Gazebo 센서와 /clock 은 브리지가 필요하다.

11. Launch 구성

simulation.launch.py 는 다음 순서로 구성한다.

1. Xacro 를 평가하여 robot_description 생성
2. robot_state_publisher 실행(use_sim_time=true)
3. Gazebo 월드 실행
4. robot_description 토픽으로 차량 스폰
5. ros_gz_bridge 실행
6. joint_state_broadcaster 활성화
7. ackermann_steering_controller 활성화
8. 선택적으로 RViz 실행

핵심 Launch 노드 예시

```
robot_state_publisher = Node(
    package='robot_state_publisher',
    executable='robot_state_publisher',
    parameters=[{
        'robot_description': robot_description,
        'use_sim_time': True,
    }],
)

spawn_robot = Node(
```

```

package='ros_gz_sim',
executable='create',
arguments=[
    '-name', 'ackermann_car',
    '-topic', 'robot_description',
    '-z', '0.20',
],
output='screen',
)

joint_state_spawner = Node(
    package='controller_manager',
    executable='spawner',
    arguments=[
        'joint_state_broadcaster',
        '--controller-manager', '/controller_manager',
    ],
)

ackermann_spawner = Node(
    package='controller_manager',
    executable='spawner',
    arguments=[
        'ackermann_steering_controller',
        '--controller-manager', '/controller_manager',
    ],
)

```

시작 순서

컨트롤러 spawner가 controller_manager 생성 전에 실행되면 연결을 재시도하거나 실패할 수 있다. TimerAction 또는 이벤트 핸들러로 스폰 이후 컨트롤러를 순차 실행하면 초기화가 더 안정적이다.

12. 단계별 검증

처음부터 SLAM 까지 한 번에 실행하지 않는다. 아래 단계를 하나씩 통과시켜야 문제 위치를 빠르게 좁힐 수 있다.

1 단계 - URDF 검사

```

cd ~/ackermann_ws

xacro \
  src/ackermann_car_description/urdf/ackermann_car.urdf.xacro \
  > /tmp/ackermann_car.urdf

check_urdf /tmp/ackermann_car.urdf

```

- 차량 전방이 +X 인지 확인
- 왼쪽이 +Y, 위쪽이 +Z 인지 확인
- 조향축이 킹핀 중심을 통과하는지 확인
- 앞바퀴가 각 너클 링크의 자식인지 확인

2 단계 - Gazebo 정지 물리

- 차량이 바닥으로 떨어지거나 공중으로 튀지 않음
- 명령 없이 바퀴와 차체가 심하게 떨리지 않음
- 차체 충돌체가 바닥에 닿지 않음

3 단계 - 컨트롤러 상태

```

ros2 control list_controllers
ros2 control list_hardware_interfaces

```

정상 상태 예시

```

joint_state_broadcaster      active
ackermann_steering_controller active

rear_right_wheel_joint/velocity    claimed
rear_left_wheel_joint/velocity     claimed
front_right_steering_joint/position claimed
front_left_steering_joint/position claimed

```

4 단계 - 직접 주행

직진 명령

```
ros2 topic pub --rate 10 \
/cmd_vel \
geometry_msgs/msg/TwistStamped \
"header:
  frame_id: base_footprint
twist:
  linear:
    x: 0.5
  angular:
    z: 0.0"
```

좌회전 명령

```
ros2 topic pub --rate 10 \
/cmd_vel \
geometry_msgs/msg/TwistStamped \
"header:
  frame_id: base_footprint
twist:
  linear:
    x: 0.4
  angular:
    z: 0.5"
```

키보드 조종

```
ros2 run teleop_twist_keyboard teleop_twist_keyboard \
--ros-args \
-p stamped:=true \
-p frame_id:=base_footprint \
-r cmd_vel:=/cmd_vel
```

angular.z 의 의미

Ackermann 컨트롤러 입력의 angular.z 는 개별 바퀴 조향각이 아니라 차량의 목표 yaw rate 다. 컨트롤러가 wheelbase 와 track width 를 사용해 좌우 조향각과 구동 바퀴 속도를 계산한다.

13. 오도메트리와 TF

```
ros2 topic echo \
/ackermann_steering_controller/odometry \
--once

ros2 run tf2_ros tf2_echo odom base_footprint
ros2 run tf2_tools view_frames
```

SLAM 실행 전 정상 TF

```
odom
|-- base_footprint
    |-- base_link
        |-- lidar_link
```

SLAM 실행 후 정상 TF

```
map
|-- odom
    |-- base_footprint
        |-- base_link
            |-- lidar_link
```

LIDAR 토픽 점검

```
gz topic -l | grep scan
ros2 topic list | grep scan
ros2 topic hz /scan
ros2 topic echo /scan --once
```

/scan 메시지의 header.frame_id 는 lidar_link 여야 하고, 같은 타임스탬프에서 odom 에서 lidar_link 까지 TF 를 계산할 수 있어야 한다.

14. SLAM Toolbox

config/slam.yaml - 핵심 설정

```
slam_toolbox:
  ros__parameters:
    use_sim_time: true

    solver_plugin: solver_plugins::CeresSolver
    ceres_linear_solver: SPARSE_NORMAL_CHOLESKY
    ceres_preconditioner: SCHUR_JACOBI
    ceres_trust_strategy: LEVENBERG_MARQUARDT
```

```

ceres_dogleg_type: TRADITIONAL_DOGLEG
ceres_loss_function: None

mode: mapping

map_frame: map
odom_frame: odom
base_frame: base_footprint
scan_topic: /scan

transform_publish_period: 0.02
map_update_interval: 2.0
transform_timeout: 0.2
tf_buffer_duration: 30.0

resolution: 0.05
min_laser_range: 0.10
max_laser_range: 12.0

minimum_time_interval: 0.1
throttle_scans: 1

use_scan_matching: true
use_scan_barycenter: true

minimum_travel_distance: 0.10
minimum_travel_heading: 0.10

scan_buffer_size: 10
scan_buffer_maximum_scan_distance: 10.0

do_loop_closing: true
loop_search_maximum_distance: 3.0

```

SLAM 실행

```

ros2 launch slam_toolbox online_async_launch.py \
  use_sim_time:=true \
  slam_params_file:=$(ros2 pkg prefix --share ackermann_car_description)/config/slam.yaml

```

프레임/토픽	설정값	발행 또는 제공 주체
map_frame	map	SLAM Toolbox
odom_frame	odom	Ackermann controller
base_frame	base_footprint	차량 기준 프레임
scan_topic	/scan	ros_gz_bridge
map -> odom	TF	SLAM Toolbox
odom -> base_footprint	TF	Ackermann controller

15. RViz 및 지도 작성

RViz 표시 항목

Display	설정
Global Options	Fixed Frame = map
Map	Topic = /map
LaserScan	Topic = /scan
RobotModel	Description Topic = /robot_description
TF	전체 프레임 연결 확인
Odometry	Ackermann controller odometry 토픽

Ackermann 차량 SLAM 주행 요령

1. 처음에는 0.2-0.5 m/s 정도로 천천히 직진한다.

2. 제자리 회전이 불가능하므로 넓은 반경으로 회전한다.
3. 벽과 약 0.5-1.5 m 거리를 유지하며 특징을 충분히 관측한다.
4. 처음 위치 주변으로 되돌아와 loop closure 가 발생하도록 한다.
5. 같은 장소를 다른 방향에서도 다시 관측한다.

```
권장 초기 명령 범위
linear.x : 0.2 - 0.5 m/s
angular.z : -0.5 - 0.5 rad/s
```

급가속 금지

Gazebo 에서 바퀴 속도가 급격히 바뀌면 미끄러짐이 커지고 wheel odometry 가 실제 이동과 달라질 수 있다. 처음에는 낮은 속도와 완만한 조향으로 기하 파라미터를 검증한다.

16. 지도 저장

```
mkdir -p ~/maps
ros2 run nav2_map_server map_saver_cli \
-f ~/maps/ackermann_slam_map
```

생성 파일

```
ackermann_slam_map.yaml
ackermann_slam_map.pgm
```

SLAM Toolbox pose graph 를 저장하면 나중에 지도를 이어서 작성하거나 localization 모드에서 사용할 수 있다. 사용 가능한 서비스 이름은 실행 중인 버전에 따라 다를 수 있으므로 먼저 다음 명령으로 확인한다.

```
ros2 service list | grep slam
```

17. 오류 진단

증상	가능한 원인	우선 확인
차량이 떨고 전진하지 않음	좌우 바퀴 축 방향, 충돌체 겹침	직진 명령에서 후륜 회전 방향과 원통 축 확인
양의 angular.z 에서 우회전	조향 joint axis 또는 조인트 순서 오류	steering_joints_names 가 right, left 순서인지 확인
컨트롤러 inactive	controller_manager 초기화 또는 인터페이스 불일치	list_hardware_interfaces 와 spawner 로그 확인
/scan 없음	Sensors 플러그인 또는 bridge 누락	gz topic -l, bridge.yaml 확인
/map 없음	scan, odom, TF, sim time 중 하나 누락	/scan, /clock, odom->base, base->lidar 순서로 확인
Message Filter dropping	scan 시간에 맞는 TF 없음	모든 노드 use_sim_time=true, frame_id 와 /clock 확인
지도와 차량 이동이 다름	wheelbase, track width, radius 또는 미끄러짐	CAD 치수 재측정 후 저속 주행

/map 이 나오지 않을 때 진단 순서

```
ros2 topic hz /scan
ros2 topic hz /clock
ros2 topic hz /ackermann_steering_controller/odometry

ros2 run tf2_ros tf2_echo odom base_footprint
ros2 run tf2_ros tf2_echo base_link lidar_link
```

최소 SLAM 입력 조건

/scan 이 지속 발행되고, odom -> base_footprint 와 base_footprint -> lidar_link 가 연결되며, /scan 의 frame_id 가 lidar_link 이고, 모든 관련 노드가 Gazebo /clock 을 사용하면 SLAM Toolbox 가 지도를 생성할 최소 조건이 갖춰진다.

18. 최종 체크리스트

분류	완료 조건
CAD	STL 7 개 분리 또는 동일한 자유도 구조 확보
CAD	wheelbase, 전/후륜 윤거, 바퀴 반지름 측정
URDF	X 전방, Y 왼쪽, Z 위쪽 좌표계 통일
URDF	앞바퀴가 너클의 자식 링크
URDF	차체/바퀴 collision 단순화
URDF	모든 동적 링크에 질량과 관성 설정
Control	후륜 velocity, 조향 position 인터페이스
Control	right, left 조인트 순서 확인
Gazebo	정지 상태에서 차량이 안정적
Drive	직진, 좌회전, 우회전 정상
Odometry	odom -> base_footprint TF 변화
LiDAR	/scan 과 lidar_link frame 확인
Time	/clock bridge 와 use_sim_time=true
SLAM	map -> odom TF 와 /map 생성
Save	map_saver_cli 로 yaml/pgm 저장

권장 구현 순서

1. Fusion/CAD 에서 STL 분리
2. 차량 기하 치수 측정
3. Xacro 링크/조인트 작성
4. RViz 에서 모델 검증
5. collision/mass/inertia 추가
6. Gazebo 정지 물리 검증
7. gz_ros2_control 연결
8. joint_state_broadcaster 활성화
9. Ackermann 컨트롤러 활성화
10. 직진 테스트
11. 좌우 조향 테스트
12. 오도메트리와 TF 확인
13. GPU LiDAR 추가

14. ros_gz_bridge 로 /scan 연결
15. RViz LaserScan 확인
16. SLAM Toolbox 실행
17. map -> odom 확인
18. 저속 주행으로 지도 작성
19. 지도 저장

19. 참고자료

1. Gazebo - ROS 2 설치 조합: https://gazebo-sim.org/docs/latest/ros_installation/
2. gz_ros2_control Jazzy 문서: https://control.ros.org/jazzy/doc/gz_ros2_control/doc/index.html
3. Ackermann Steering Controller:
https://control.ros.org/jazzy/doc/ros2_controllers/ackermann_steering_controller/doc/userdoc.html
4. Steering Controllers Library:
https://control.ros.org/jazzy/doc/ros2_controllers/steering_controllers_library/doc/userdoc.html
5. ROS 2 URDF 튜토리얼: <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/URDF/URDF-Main.html>
6. Gazebo 센서 문서: <https://gazebo-sim.org/docs/latest/sensors/>
7. Gazebo ROS 2 통합: https://gazebo-sim.org/docs/latest/ros2_integration/
8. SLAM Toolbox 저장소: https://github.com/SteveMacenski/slam_toolbox

마지막 메모

가장 중요한 순서는 직진 -> 조향 -> 오도메트리 -> LiDAR -> TF -> SLAM 이다. 한 번에 모든 기능을 붙이지 말고 각 단계에서 토픽, 조인트, TF 를 확인하면 문제 해결 시간이 크게 줄어든다.